

Estructuras Discretas II

Dr(c) Wilber Ramos Lovón

2026

Prólogo

¿Por qué este libro?

Cuando un estudiante de Ciencia de la Computación se enfrenta por primera vez a las matemáticas discretas, suele hacer dos preguntas fundamentales:

- ¿para qué sirve esto?
- ¿cómo se relaciona con la programación que ya conozco?

Este libro es mi intento de responder a esas preguntas de la manera más clara y práctica posible.

Estructura del libro

Este libro es la continuación natural de *Estructuras Discretas I* y está organizado en cinco partes principales:

1. **Combinatoria:** El arte del conteo.
2. **Teoría de Grafos:** El estudio de redes y relaciones.
3. **Criptografía:** La ciencia de la comunicación segura.
4. **Programación Funcional con Haskell:** Una introducción práctica.
5. **Guías de Práctica:** Ejercicios y proyectos.

¿Por qué Haskell?

Haskell no es solo otro lenguaje de programación. Es una herramienta que permite:

- Escribir funciones que son matemáticamente puras.
- Razonar sobre el código mediante demostraciones formales.
- Implementar estructuras de datos con elegancia matemática.
- Verificar experimentalmente los conceptos teóricos.

Agradecimientos

A los estudiantes de la Escuela Profesional de Ciencia de la Computación.

Wilber Roberto Ramos Lovón
Arequipa, 2026

Índice general

Prólogo	II
I Técnicas Avanzadas de Conteo	1
1. Combinatoria	2
1.1. Principio de la multiplicación	2
1.2. Principio de la Suma	3
2. Relaciones de Recurrencia	5
2.1. Definiciones	5
2.2. Solución de Recurrencias	5
2.3. Recurrencias Homogéneas Lineales	6
2.4. Recurrencias No Homogéneas	6
3. Funciones Generatrices	7
3.1. Funciones generatrices	7
4. Análisis de Algoritmos	9
4.1. Introducción	9
4.2. Notación Asintótica	9
II Teoría de Grafos	12
5. Teoría de Grafos	13
5.1. Conceptos fundamentales	13
5.1.1. Definiciones básicas	13
5.2. Representación numérica	16

5.2.1.	Matriz de adyacencia	16
5.2.2.	Lista de adyacencia	16
5.3.	Relaciones de adyacencia	16
5.4.	Vecindades	16
5.5.	Trayectorias	17
5.6.	Grafos conexos	17
5.7.	Algunos resultados	18
5.7.1.	Algoritmo de Fleury	18
5.7.2.	Caminos Hamiltonianos	18
5.8.	Ejercicios	19
6.	Conexión y Recorridos	20
6.1.	Trayectorias y Caminos	20
6.1.1.	Tipos de caminos	20
6.2.	Grafos Conexos	20
6.3.	Recorridos en Grafos	22
6.3.1.	Búsqueda en profundidad (DFS)	22
6.3.2.	Búsqueda en anchura (BFS)	22
6.4.	Caminos Eulerianos y Hamiltonianos	22
6.5.	Implementación en Haskell	22
6.6.	Ejercicios	23
7.	Árboles	24
7.1.	Definiciones Básicas	24
7.2.	Clasificación	24
7.2.1.	Árboles binarios	24
7.3.	Árboles Ponderados	25
7.3.1.	Árboles generadores	25
7.3.2.	Códigos de Huffman	25
7.4.	Implementación en Haskell	25
7.5.	Ejercicios	26
8.	Algoritmos sobre Grafos	27
8.1.	Camino más Corto	27
8.1.1.	Algoritmo de Dijkstra	27
8.2.	Árboles Generadores Mínimos	29
8.3.	Redes de Transporte	29
8.4.	Ejercicios	29

III	Criptografía	30
9.	Aritmética Modular	31
9.1.	Introducción	31
9.2.	Operaciones Modulares	31
9.3.	Algoritmo de Euclides	31
9.4.	Inversos Modulares	31
9.5.	Pequeño Teorema de Fermat	32
9.6.	Implementación en Haskell	32
9.7.	Ejercicios	33
10.	Criptografía Clásica	34
10.1.	Introducción	34
10.2.	Cifrado César	34
10.3.	Cifrado de Sustitución	34
10.4.	Cifrado de Vigenère	34
10.5.	Implementación en Haskell	35
10.6.	Ejercicios	35
11.	Criptografía RSA	36
11.1.	Introducción	36
11.2.	Generación de Claves	36
11.3.	Cifrado y Descifrado	36
11.4.	Seguridad	36
11.5.	Implementación en Haskell	37
11.6.	Ejercicios	38
IV	Programación Funcional con Haskell	39
12.	Introducción a Haskell	40
12.1.	¿Qué es Haskell?	40
12.1.1.	Características principales	40
12.2.	Tipos Básicos	41
12.2.1.	Enteros	41
12.2.2.	Reales	41
12.2.3.	Bool, Char, String	41
12.3.	Definición de funciones	41

12.4. Listas	41
12.5. Funciones de Orden Superior	42
12.6. Tuplas	42
12.7. Ejercicios	42
13.Haskell para Combinatoria	43
13.1. Generación de Permutaciones	43
13.2. Números Especiales	44
14.Haskell para Grafos	45
14.1. Representación de Grafos	45
14.2. Algoritmos de Recorrido	45
15.Haskell para Criptografía	47
15.1. Aritmética Modular	47
15.2. RSA Completo	48
V Guías de Práctica	49
16.Guía de Práctica 1: Técnicas Avanzadas de Conteo	50
16.1. Objetivos	50
16.2. Principios Básicos de Combinatoria	50
16.3. Relaciones de Recurrencia	51
16.4. Funciones Generatrices	51
16.5. Análisis de Algoritmos	51
16.6. Problemas Integradores	52
16.7. Referencias	52
17.Práctica: Grafos en Haskell	53
17.1. Objetivos	53
17.2. Ejercicios de Implementación	53
17.2.1. Representación y Recorridos	53
17.2.2. Algoritmos Avanzados	53
17.3. Proyecto	54
18.Práctica: Criptografía en Haskell	55
18.1. Objetivos	55
18.2. Ejercicios de Implementación	55

18.2.1. Cifrados Clásicos	55
18.2.2. RSA	55
18.3. Proyecto	56

Parte I

Técnicas Avanzadas de Conteo

Capítulo 1

Combinatoria

Aunque la enumeración o conteo se percibe como un concepto básico de la aritmética, su importancia y complejidad suelen subestimarse. Es esencial comprender los principios fundamentales en lugar de confiar únicamente en fórmulas.

1.1. Principio de la multiplicación

Si una actividad se puede realizar en t pasos sucesivos, y el paso i se puede hacer de n_i maneras, entonces el número total de actividades es $n_1 \cdot n_2 \cdots n_t$.

Ejemplo 1.1. En una picantería se ofrece: 2 entradas, 3 platos de fondo y 4 bebidas. ¿Cuántas formas hay de seleccionar un plato de fondo y una bebida?

$$\text{Respuesta} = 3 \cdot 4 = 12.$$

¿Y una entrada, un plato y una bebida?

$$\text{Respuesta} = 2 \cdot 3 \cdot 4 = 24.$$

Ejemplo 1.2. ¿Cuántas cadenas de longitud 4 se pueden formar con las letras ABCDE sin repetición?

- a) Total: $5 \cdot 4 \cdot 3 \cdot 2 = 120$.
- b) Comienzan con B: $1 \cdot 4 \cdot 3 \cdot 2 = 24$.
- c) No comienzan con B: $120 - 24 = 96$.

Ejemplo 1.3. ¿Cuántos subconjuntos tiene $A = \{a_1, \dots, a_n\}$?

$$\underbrace{2 \cdot 2 \cdots 2}_{n \text{ factores}} = 2^n.$$

1.2. Principio de la Suma

Si $X_1, \dots, X_t$ son conjuntos ajenos por pares con n_i elementos cada uno, el número de elementos en la unión es $n_1 + \dots + n_t$.

Ejemplo 1.4. ¿Cuántas cadenas de ocho bits comienzan con 101 o con 111?

$$2^5 + 2^5 = 64.$$

Definición 1.5. Una **permutación** de n elementos distintos es un ordenamiento de los n elementos.

Teorema 1.6. Existen $n!$ permutaciones de n elementos.

Demostración. Por el principio de la multiplicación: $n(n-1)(n-2) \cdots 3 \cdot 2 \cdot 1 = n!$. $\square$

Definición 1.7. Una **permutación** r de n elementos es un ordenamiento de r de ellos. Se denota $P(n, r)$.

Teorema 1.8.

$$P(n, r) = \frac{n!}{(n-r)!}, \quad r \leq n.$$

Ejemplo 1.9. ¿De cuántas maneras se puede elegir presidente, vicepresidente, secretario y tesorero entre 10 delegados?

$$P(10, 4) = 10 \cdot 9 \cdot 8 \cdot 7 = 5040.$$

Definición 1.10. Una **combinación** r de X es una selección no ordenada de r elementos de X . Se denota $C(n, r)$ o $\binom{n}{r}$.

Teorema 1.11.

$$C(n, r) = \frac{n!}{(n-r)! r!}.$$

Ejemplo 1.12. ¿De cuántas maneras elegir 3 representantes entre 5 delegados?

$$C(5, 3) = \frac{5!}{2! 3!} = 10.$$

Ejemplo 1.13. Comité de 2 mujeres y 3 hombres de 5 mujeres y 6 hombres:

$$C(5, 2) C(6, 3) = 10 \cdot 20 = 200.$$

Ejemplo 1.14. ¿Cuántas cadenas de 8 bits contienen exactamente 4 unos?

$$C(8, 4) = 70.$$

Ejemplo 1.15. Baraja de 52 cartas:

- a) Manos de póquer: $C(52, 5) = 2\,598\,960$.
- b) Todas del mismo palo: $4 \cdot C(13, 5) = 5148$.
- c) Tres de una denominación y dos de otra: $13 \cdot 12 \cdot C(4, 3) \cdot C(4, 2) = 3744$.

Capítulo 2

Relaciones de Recurrencia

2.1. Definiciones

Definición 2.1. Una **relación de recurrencia** para la sucesión $a_0, a_1, a_2, \dots$ es una ecuación que relaciona a_n con predecesores. Las **condiciones iniciales** son valores dados explícitamente para un número finito de términos.

Ejemplo 2.2 (Fibonacci).

$$f_n = f_{n-1} + f_{n-2}, \quad n \geq 3, \quad f_1 = f_2 = 1.$$

Ejemplo 2.3 (Torres de Hanoi).

$$h_n = 2h_{n-1} + 1, \quad h_1 = 1.$$

2.2. Solución de Recurrencias

Definición 2.4. Resolver una relación de recurrencia significa hallar una fórmula explícita para a_n .

Ejemplo 2.5. $a_n = a_{n-1} + 3, a_1 = 2 \Rightarrow a_n = 2 + 3(n - 1)$.

Ejemplo 2.6. $h_n = 2h_{n-1} + 1, h_1 = 1 \Rightarrow h_n = 2^n - 1$.

Ejemplo 2.7. $a_n = 3n a_{n-1}, a_0 = 2 \Rightarrow a_n = 2 \cdot 3^n n!$.

2.3. Recurrencias Homogéneas Lineales

Definición 2.8. Una recurrencia homogénea lineal de orden k con coeficientes constantes es de la forma

$$a_n = c_1 a_{n-1} + \cdots + c_k a_{n-k}, \quad c_k \neq 0.$$

Teorema 2.9. Para la recurrencia de orden 2, $a_n = c_1 a_{n-1} + c_2 a_{n-2}$:

1. Si U_n, V_n son soluciones, $bU_n + dV_n$ también lo es.
2. Si r es raíz de $t^2 - c_1 t - c_2 = 0$, entonces r^n es solución.
3. Si $r_1 \neq r_2$ reales: $a_n = br_1^n + dr_2^n$.
4. Si r es raíz doble: $a_n = br^n + dnr^n$.
5. Si las raíces son complejas: solución trigonométrica.

Ejemplo 2.10 (Fibonacci). $f_n = f_{n-1} + f_{n-2}$, $f_0 = 0$, $f_1 = 1$. La ecuación característica es $r^2 - r - 1 = 0$, cuyas raíces son $\frac{1 \pm \sqrt{5}}{2}$. Por tanto,

$$f_n = \frac{1}{\sqrt{5}} \left(\frac{1 + \sqrt{5}}{2} \right)^n - \frac{1}{\sqrt{5}} \left(\frac{1 - \sqrt{5}}{2} \right)^n.$$

Ejemplo 2.11. $a_n = 6a_{n-1} - 9a_{n-2}$, $a_0 = 1$, $a_1 = -1$. La ecuación característica da $r = 3$ doble. Así $a_n = (b + dn)3^n$. Con las condiciones iniciales, $b = 1$, $d = -4/3$. Por tanto,

$$a_n = \left(1 - \frac{4}{3}n\right) 3^n.$$

2.4. Recurrencias No Homogéneas

Teorema 2.12. Dada $a_n = c_1 a_{n-1} + c_2 a_{n-2} + f(n)$, si $g(n)$ es solución particular y V_n es solución de la homogénea asociada, entonces $U_n = V_n + g(n)$ es solución general.

Ejemplo 2.13. $a_n = 6a_{n-1} - 8a_{n-2} + 3$, $a_0 = 0$, $a_1 = 1$. Homogénea: $r^2 - 6r + 8 = 0 \Rightarrow r = 2, 4$. Solución particular constante: $A = 6A - 8A + 3 \Rightarrow A = -1$. Con las condiciones iniciales, $b = 0$, $d = 1/2$. Luego,

$$a_n = \frac{1}{2} 4^n - 1.$$

Capítulo 3

Funciones Generatrices

3.1. Funciones generatrices

Definición 3.1. La **función generatriz** de la sucesión $a_0, a_1, \dots$ es

$$f(x) = \sum_{i=0}^{\infty} a_i x^i.$$

Ejemplo 3.2. $(1+x)^n$ es la función generatriz de $\binom{n}{0}, \binom{n}{1}, \dots, \binom{n}{n}, 0, 0, \dots$

Ejemplo 3.3. $\frac{1}{1-x} = 1 + x + x^2 + x^3 + \dots$ es la función generatriz de $1, 1, 1, \dots$

Ejemplo 3.4. $\frac{1}{(1-x)^2} = 1 + 2x + 3x^2 + \dots$ es la función generatriz de $1, 2, 3, \dots$

Ejemplo 3.5. La función generatriz de $0^2, 1^2, 2^2, \dots$ es $\frac{x(1+x)}{(1-x)^3}$.

Ejemplo 3.6. Mónica compró 12 naranjas para Graciela, María y Francisco. Graciela recibe al menos 4, María al menos 2 y Francisco entre 2 y 5. La función generatriz es:

$$f(x) = (x^4 + \dots + x^8)(x^2 + \dots + x^6)(x^2 + \dots + x^5).$$

El coeficiente de x^{12} es 14.

Cuadro 3.1: Funciones generatrices fundamentales

No.	Función generatriz
1	$(1+x)^m = \sum_{n=0}^m \binom{m}{n} x^n$
2	$(1+ax)^m = \sum_{n=0}^m \binom{m}{n} a^n x^n$
3	$(1+x^m)^n = \sum_{k=0}^n \binom{n}{k} x^{mk}$
4	$\frac{1-x^{m+1}}{1-x} = 1+x+\cdots+x^m$
5	$\frac{1}{1-x} = \sum_{n=0}^{\infty} x^n$
6	$\frac{1}{(1-x)^m} = \sum_{n=0}^{\infty} \binom{m+n-1}{n} x^n$
7	$\frac{1}{(1+x)^m} = \sum_{n=0}^{\infty} (-1)^n \binom{m+n-1}{n} x^n$
8	$e^x = \sum_{n=0}^{\infty} \frac{x^n}{n!}$
9	$\ln(1+x) = \sum_{n=1}^{\infty} (-1)^{n+1} \frac{x^n}{n}$
10	$\frac{1}{(1-x)^2} = \sum_{n=0}^{\infty} (n+1)x^n$

Capítulo 4

Análisis de Algoritmos

4.1. Introducción

Definición 4.1. La **demora temporal** de un algoritmo es una medida que estima el tiempo que tarda un algoritmo en completarse en función del tamaño de la entrada.

Observación 4.2. El tiempo se mide en operaciones fundamentales, no en segundos. Depende de la entrada:

- **Mejor caso:** tiempo mínimo.
- **Peor caso:** tiempo máximo.
- **Caso promedio:** promedio.

4.2. Notación Asintótica

Definición 4.3. Sean $T, g : \mathbb{Z}^+ \rightarrow \mathbb{R}$.

- $T(n) = O(g(n))$ si $\exists C_1 > 0$ tal que $|T(n)| \leq C_1|g(n)|$ para casi todo n .
- $T(n) = \Omega(g(n))$ si $\exists C_2 > 0$ tal que $|T(n)| \geq C_2|g(n)|$ para casi todo n .
- $T(n) = \Theta(g(n))$ si $T(n) = O(g(n))$ y $T(n) = \Omega(g(n))$.

Ejemplo 4.4. Para $T(n) = 35n^2 + 7n + 4$:

$$T(n) \leq 46n^2 \Rightarrow T(n) = O(n^2), \quad T(n) \geq 35n^2 \Rightarrow T(n) = \Omega(n^2).$$

Por tanto, $T(n) = \Theta(n^2)$.

Observación 4.5. Si $T(n) = a_k n^k + \dots + a_0$, entonces $T(n) = \Theta(n^k)$.

Ejemplo 4.6 (Búsqueda lineal).

```

1 def busquedaLineal(lista, elemento):
2     n = len(lista)
3     i = 0
4     while i < n and lista[i] != elemento:
5         i = i + 1
6     return i if i < n else print("no encontrado")

```

Peor caso: $T(n) = \Theta(n)$.

Ejemplo 4.7 (Multiplicación de matrices).

```

1 def multiplicacion_matrices(A, B):
2     n = len(A)
3     C = [[0 for _ in range(n)] for _ in range(n)]
4     for i in range(n):
5         for j in range(n):
6             for k in range(n):
7                 C[i][j] += A[i][k] * B[k][j]
8     return C

```

$T(n) = n \cdot n^2 = n^3$, por lo que $T(n) = \Theta(n^3)$.

Ejemplo 4.8 (Bucle anidado).

```

1 def contarIteraciones(n):
2     x = 0
3     for i in range(1, n + 1):
4         for j in range(1, i + 1):
5             x += 1
6     return x

```

$T(n) = 1 + 2 + \dots + n = \Theta(n^2)$.

Ejemplo 4.9 (Subsets).

```

1 def generaSubsets(elements):
2     n = len(elements)
3     subsets = []

```

```

4   for i in range(1 << n):
5       subset = [elements[j] for j in range(n) if (i >>
6           j) & 1]
7       subsets.append(subset)
   return subsets

```

$$T(n) = \Theta(n \cdot 2^n).$$

Observación 4.10. En la práctica se buscan complejidades $\Theta(1)$, $\Theta(\log n)$, $\Theta(n)$ o $\Theta(n \log n)$. Las de tipo $\Theta(2^n)$, $\Theta(n!)$ son intratables para entradas moderadas.

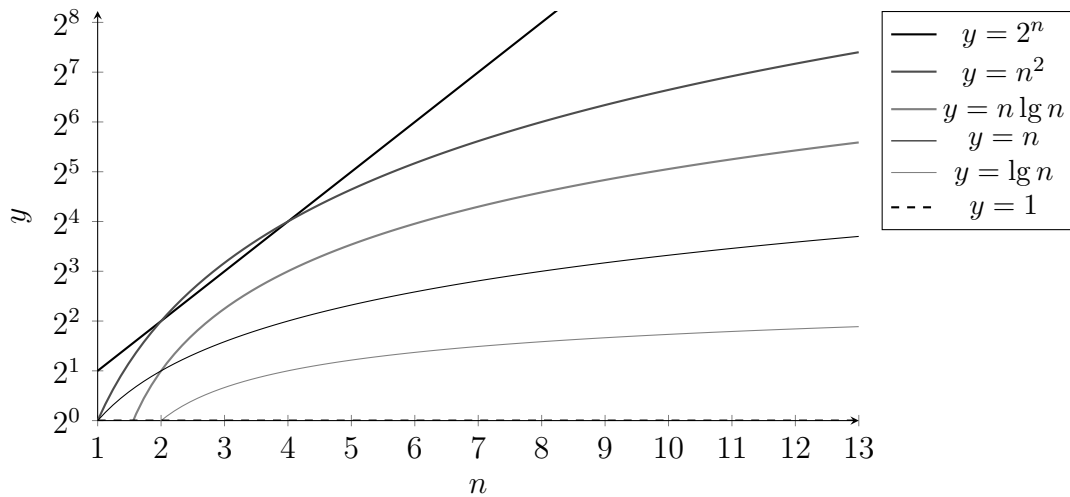


Figura 4.1: Crecimiento de funciones comunes en complejidad computacional.

Ejemplo 4.11 (Bucle con división).

```

1   j = n
2   while (j >= 1):
3       for i = 1 to j: x = x + 1
4       j = floor(j/2)

```

$$T(n) = n + \lfloor n/2 \rfloor + \dots \leq 2n, \text{ luego } T(n) = \Theta(n).$$

Observación 4.12. Un problema es **tratable** si admite un algoritmo polinomial. Es **intratable** si solo se conocen algoritmos exponenciales o factoriales. Es **irresoluble** si no existe algoritmo (por ejemplo, el problema de la parada).

Parte II

Teoría de Grafos

Capítulo 5

Teoría de Grafos

5.1. Conceptos fundamentales

5.1.1. Definiciones básicas

Definición 5.1. Una **familia enumerable** es una colección enumerable de objetos iguales o diferentes. El mayor número p de elementos iguales es el **grado de multiplicidad**. Si $p = 1$ se llama **conjunto**.

Definición 5.2. Un **grafo** es una estructura $G = (N, A)$ donde N es un conjunto finito y A una familia de elementos no vacíos de $\mathcal{P}(N)$.

- Si los elementos de A son subconjuntos de N , G es un **hipergrafo no orientado**.
- Si son pares ordenados, G es un **hipergrafo orientado**.
- Los elementos de N son **nodos**; $n = |N|$ es el **orden**.
- Si las conexiones son de 2 nodos no orientadas, son **aristas**; si son orientadas, **arcos**.
- $m = |A|$ es el **tamaño** del grafo.

Ejemplo 5.3. La Figura 5.1 muestra aristas, arcos, hiper-aristas e hiper-arcos.

Definición 5.4. Dos grafos $G_1 = (N_1, A_1)$ y $G_2 = (N_2, A_2)$ son:

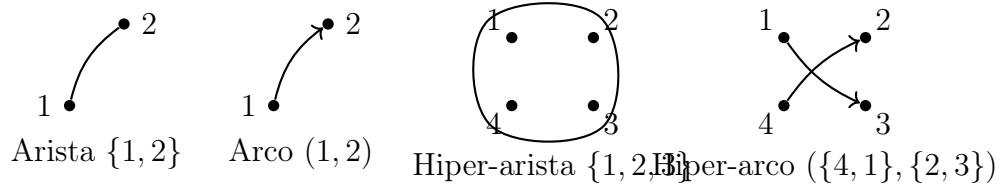


Figura 5.1: Aristas, arcos, hiper-aristas e hiper-arcos.

1. **Iguales** si $N_1 = N_2$ y $A_1 = A_2$.
2. **Isomorfos** si existe biyección $f : N_1 \rightarrow N_2$ que preserve adyacencia.

Ejemplo 5.5. En la Figura 5.2 se muestran dos grafos isomorfos.

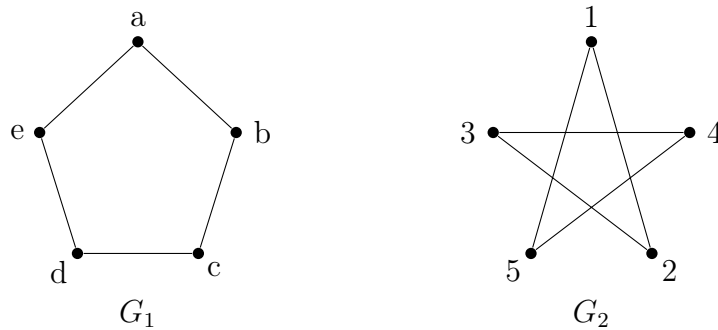


Figura 5.2: Los grafos G_1 y G_2 son isomorfos.

Definición 5.6. Un grafo $G = (N, A)$ es **k -partido** si N se puede particionar en $N_1, \dots, N_k$ sin aristas dentro de cada N_i .

Ejemplo 5.7. Ver Figura 5.3.

Ejemplo 5.8 (Puentes de Königsberg). Euler representó las zonas de Königsberg y sus puentes con el multigrafo de la Figura 5.4, dando inicio a la Teoría de Grafos.

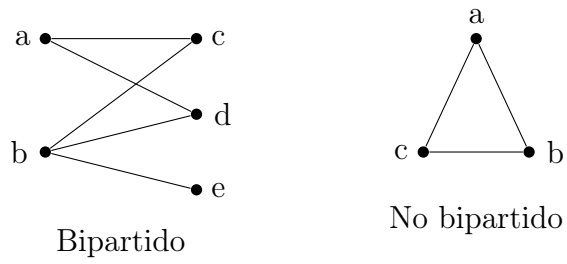


Figura 5.3: Grafo bipartido y no bipartido.

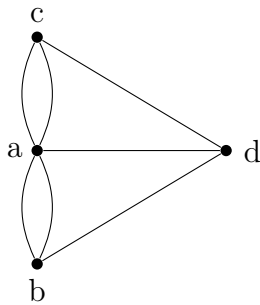


Figura 5.4: Multigrafo de los 7 puentes de Königsberg.

5.2. Representación numérica

5.2.1. Matriz de adyacencia

Dado un grafo orientado G con nodos $v_1, \dots, v_n$:

$$[A]_{ij} = \begin{cases} 1 & \text{si } (v_i, v_j) \in A \\ 0 & \text{si no} \end{cases}$$

Ejemplo 5.9. Para un dígrafo de 5 nodos con lazos en 1 y 2:

$$A = \begin{bmatrix} 1 & 0 & 0 & 0 & 1 \\ 0 & 1 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 \\ 1 & 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \end{bmatrix}$$

5.2.2. Lista de adyacencia

Es más económica: $n + m$ posiciones en memoria.

5.3. Relaciones de adyacencia

Definición 5.10. En un grafo orientado $G = (N, A)$, y es **sucesor** de x si $(x, y) \in A$. Dualmente, x es **antecesor** de y .

Definición 5.11. Un nodo y es **alcanzable** desde x si existe una secuencia de sucesores que va de x a y .

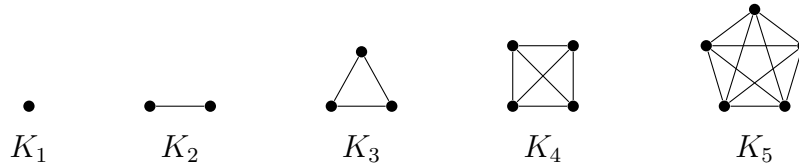
5.4. Vecindades

Definición 5.12. El **grado** $\delta(x)$ de un nodo es el número total de conexiones que inciden en él.

Definición 5.13. Un grafo G es **completo** si cada par de nodos tiene una conexión. Se denota K_n .

Ejemplo 5.14. Ver Figura 5.5.

Definición 5.15. El **complemento** $\overline{G}$ tiene los mismos nodos y las conexiones que no están en G .

Figura 5.5: Grafos completos K_1 a K_5 .

5.5. Trayectorias

Definición 5.16. Un **recorrido** (o cadena) es una familia de conexiones sucesivamente adyacentes. Es **cerrado** si la última es adyacente a la primera; **abierto** en caso contrario. Es **simple** si no repite conexiones; **elemental** si no repite nodos. Un **ciclo** es una cadena simple y cerrada. Un **camino** es una cadena orientada. Un **circuito** es un camino simple y cerrado.

Definición 5.17. Una cadena es **euleriana** si recorre cada conexión exactamente una vez; **hamiltoniana** si visita cada nodo exactamente una vez. Un grafo es **euleriano** (o **hamiltoniano**) si posee la cadena cerrada correspondiente.

5.6. Grafos conexos

Definición 5.18. Un grafo es **conexo** si para todo par de nodos se puede llegar de uno al otro.

Definición 5.19. Un dígrafo es:

- **s-conexo** (simplemente conexo) si todo par de nodos está unido por una cadena.
- **sf-conexo** (semi-fuertemente conexo) si en todo par, al menos uno es alcanzable desde el otro.
- **f-conexo** (fuertemente conexo) si todo par de nodos son mutuamente alcanzables.

5.7. Algunos resultados

Teorema 5.20 (Lema del apretón de manos). *Si $G = (N, A)$ es un multigrafo, entonces*

$$\sum_{v \in N} \delta(v) = 2|A|.$$

Demostración. Cada arista contribuye con 2 unidades al total de grados. $\square$

Corolario 5.21. *En todo multigrafo, el número de nodos de grado impar es par.*

Ejemplo 5.22. ¿Existe un grafo 4-regular con 10 aristas? Si $\delta(v) = 4$ para todo v , entonces $4|N| = 20 \Rightarrow |N| = 5$. El grafo K_5 cumple (10 aristas, 5 nodos, grado 4).

Teorema 5.23 (Euler). *Un grafo no dirigido sin nodos aislados tiene un ciclo euleriano si y solo si es conexo y todos sus nodos tienen grado par.*

Demostración. $(\Rightarrow)$ Si G es euleriano, es conexo (recorre todos los nodos). Al recorrer un nodo, entramos y salimos, así que su grado es par.

$(\Leftarrow)$ Por inducción sobre $|A|$. Como todos los grados son pares, todo nodo tiene grado ≥ 2 , luego existe un ciclo. Recorrerlo, eliminarlo, y aplicar inducción sobre cada componente conexa resultante, uniéndolas por nodos comunes. $\square$

Corolario 5.24. *Existe una cadena abierta euleriana si y solo si G es conexo y tiene exactamente dos nodos de grado impar.*

Ejemplo 5.25 (Königsberg). En el multigrafo de la Figura 5.4: $\delta(a) = 5$, $\delta(b) = 3$, $\delta(c) = 3$, $\delta(d) = 3$. Todos impares, por lo que no hay ciclo euleriano.

5.7.1. Algoritmo de Fleury

5.7.2. Caminos Hamiltonianos

Teorema 5.26 (Ore). *Sea G sin lazos, $|N| = n \geq 3$. Si $\delta(x) + \delta(y) \geq n$ para todo par no adyacente, entonces G es hamiltoniano.*

Corolario 5.27 (Dirac). *Si $|N| = n \geq 3$ y $\delta(x) \geq n/2$ para todo x , entonces G es hamiltoniano.*

Algoritmo 1 Algoritmo de Fleury

Entrada: Grafo conexo $G = (N, A)$ con todos los grados pares**Salida:** Ciclo euleriano π

```

1: Elegir  $v \in N$  como nodo inicial
2:  $\pi \leftarrow (v)$ 
3: while existan aristas en  $A$  do
4:   Sea  $\pi = (v, u, \dots, w)$ 
5:   if en  $w$  solo existe una arista  $\{w, z\}$  then
6:     Extender  $\pi$  con  $z$ 
7:     Eliminar  $\{w, z\}$  de  $A$ 
8:   else
9:     Elegir arista  $\{w, z\}$  que no sea puente
10:    Extender  $\pi$  con  $z$ 
11:    Eliminar  $\{w, z\}$  de  $A$ 
12:   end if
13: end while
14: return  $\pi$ 

```

5.8. Ejercicios

Ejercicio 5.28. Los turistas Jenssen, Leuzinger, Dufour y Medeiros hablan inglés, francés, portugués y alemán. Represente con un grafo las posibilidades de comunicación.

Ejercicio 5.29. Determine el número mínimo de guardias para vigilar una sección de grandes almacenes.

Ejercicio 5.30. Demuestre que K_n tiene $\binom{n}{2}$ aristas.

Ejercicio 5.31. ¿Para qué n tiene K_n un ciclo euleriano?

Ejercicio 5.32. Sea G autocomplementario con n nodos. Demuestre que $n = 4k$ o $n = 4k + 1$.

Capítulo 6

Conexión y Recorridos

6.1. Trayectorias y Caminos

Un camino es una secuencia de vértices donde cada par consecutivo está conectado por una arista.

6.1.1. Tipos de caminos

- **Camino simple:** no repite vértices.
- **Ciclo:** comienza y termina en el mismo vértice.
- **Trayectoria:** sin aristas repetidas.

6.2. Grafos Conexos

Un grafo es conexo si existe un camino entre cualquier par de vértices. Una **componente conexa** es un subgrafo maximal conexo.

Algoritmo 2 DFS

Entrada: Grafo $G = (V, E)$, vértice inicial s

```
1: visitado[v] ← falso para todo  $v$ 
2: procedure DFS( $u$ )
3:   visitado[ $u$ ] ← verdadero
4:   for cada  $v$  adyacente a  $u$  do
5:     if no visitado[ $v$ ] then
6:       DFS( $v$ )
7:     end if
8:   end for
9: end procedure
10: DFS( $s$ )
```

Algoritmo 3 BFS

Entrada: Grafo $G = (V, E)$, vértice inicial s

```
1: visitado[v] ← falso; visitado[ $s$ ] ← verdadero
2: cola ← [ $s$ ]
3: while cola  $\neq \emptyset$  do
4:    $u \leftarrow \text{POPFront}(\text{cola})$ 
5:   for cada  $v$  adyacente a  $u$  do
6:     if no visitado[ $v$ ] then
7:       visitado[ $v$ ] ← verdadero
8:       cola ← cola + [ $v$ ]
9:     end if
10:  end for
11: end while
```

6.3. Recorridos en Grafos

6.3.1. Búsqueda en profundidad (DFS)

6.3.2. Búsqueda en anchura (BFS)

6.4. Caminos Eulerianos y Hamiltonianos

Un camino euleriano recorre todas las aristas exactamente una vez. Un grafo conexo tiene ciclo euleriano si y solo si todos los vértices tienen grado par.

Un camino hamiltoniano visita todos los vértices exactamente una vez. **Teorema de Dirac:** si $\deg(v) \geq n/2$ para todo v en un grafo con $n \geq 3$ vértices, tiene ciclo hamiltoniano.

6.5. Implementación en Haskell

Listing 6.1: DFS y BFS en Haskell

```

1 dfs :: Eq a => a -> [(a, [a])] -> [a]
2 dfs inicio grafo = dfsAux inicio []
3   where
4     dfsAux v visitados
5       | v `elem` visitados = visitados
6       | otherwise = foldl (\acc w -> dfsAux w acc) (v:
7         visitados)
8         (maybe [] id (lookup v grafo))
9
10 bfs :: Eq a => a -> [(a, [a])] -> [a]
11 bfs inicio grafo = bfsAux [inicio] []
12   where
13     bfsAux [] vis = vis
14     bfsAux (v:cola) vis
15       | v `elem` vis = bfsAux cola vis
16       | otherwise = bfsAux (cola ++ maybe [] id (lookup v
17         grafo)) (vis ++ [v])

```

6.6. Ejercicios

1. Ejecutar DFS y BFS en un grafo de ejemplo.
2. Demostrar que un grafo tiene ciclo euleriano si todos los vértices tienen grado par.
3. Implementar en Haskell un algoritmo que encuentre un camino euleriano.

Capítulo 7

Árboles

7.1. Definiciones Básicas

Un **árbol** es un grafo conexo sin ciclos. Para un grafo G con n vértices son equivalentes:

1. G es un árbol.
2. G es conexo y tiene $n - 1$ aristas.
3. G no tiene ciclos y tiene $n - 1$ aristas.
4. Entre cualquier par de vértices existe exactamente un camino.

7.2. Clasificación

7.2.1. Árboles binarios

Un árbol binario es un árbol enraizado donde cada nodo tiene a lo sumo dos hijos. Propiedades:

- Un árbol binario con n nodos tiene $n - 1$ aristas.
- La altura es al menos $\lceil \log_2(n + 1) \rceil$.
- El nivel i tiene a lo sumo 2^i nodos.

7.3. Árboles Ponderados

7.3.1. Árboles generadores

Un **árbol generador** de G es un subgrafo que es árbol y contiene todos los vértices.

7.3.2. Códigos de Huffman

Algoritmo 4 Algoritmo de Huffman

Entrada: Símbolos con frecuencias

Salida: Árbol de codificación binario

- 1: Crear nodo hoja por símbolo con su frecuencia
 - 2: Insertar todos los nodos en cola de prioridad
 - 3: **while** cola tiene más de un elemento **do**
 - 4: Extraer los dos nodos de menor frecuencia
 - 5: Crear nuevo nodo con frecuencia suma
 - 6: Los nodos extraídos son hijos del nuevo
 - 7: Insertar el nuevo en la cola
 - 8: **end while**
 - 9: **return** raíz del árbol
-

7.4. Implementación en Haskell

Listing 7.1: Árboles binarios en Haskell

```
1 data ArbolBinario a = Vacio | Nodo a (ArbolBinario a) (  
    ArbolBinario a)  
2     deriving (Show, Eq)  
3  
4 preorden :: ArbolBinario a -> [a]  
5 preorden Vacio = []  
6 preorden (Nodo x izq der) = x : preorden izq ++ preorden  
    der  
7  
8 inorden :: ArbolBinario a -> [a]
```

```
9 inorden Vacio = []
10 inorden (Nodo x izq der) = inorden izq ++ [x] ++ inorden
    der
11
12 postorden :: ArbolBinario a -> [a]
13 postorden Vacio = []
14 postorden (Nodo x izq der) = postorden izq ++ postorden
    der ++ [x]
15
16 altura :: ArbolBinario a -> Int
17 altura Vacio = 0
18 altura (Nodo _ izq der) = 1 + max (altura izq) (altura
    der)
```

7.5. Ejercicios

1. Demostrar que un árbol con n vértices tiene $n - 1$ aristas.
2. Implementar en Haskell un árbol de Huffman.
3. ¿Cuál es la altura máxima de un árbol binario con n nodos?

Capítulo 8

Algoritmos sobre Grafos

8.1. Camino más Corto

8.1.1. Algoritmo de Dijkstra

Algoritmo 5 Algoritmo de Dijkstra

Entrada: Grafo ponderado $G = (V, E)$, origen s

Salida: Distancias mínimas desde s

```
1:  $\text{dist}[v] \leftarrow \infty, \text{dist}[s] \leftarrow 0$ 
2:  $Q \leftarrow V$ 
3: while  $Q \neq \emptyset$  do
4:    $u \leftarrow \arg \min_{v \in Q} \text{dist}[v]$ 
5:    $Q \leftarrow Q \setminus \{u\}$ 
6:   for cada  $v$  adyacente a  $u$  do
7:     if  $\text{dist}[v] > \text{dist}[u] + w(u, v)$  then
8:        $\text{dist}[v] \leftarrow \text{dist}[u] + w(u, v)$ 
9:     end if
10:  end for
11: end while
12: return  $\text{dist}$ 
```

Algoritmo 6 Algoritmo de Prim

Entrada: Grafo ponderado conexo $G = (V, E)$ **Salida:** Árbol generador mínimo T

```
1:  $T \leftarrow \emptyset, S \leftarrow \{v_0\}$ 
2: while  $S \neq V$  do
3:   Encontrar  $(u, v)$  de peso mínimo con  $u \in S, v \notin S$ 
4:    $S \leftarrow S \cup \{v\}$ 
5:    $T \leftarrow T \cup \{(u, v)\}$ 
6: end while
7: return  $T$ 
```

Algoritmo 7 Algoritmo de Kruskal

Entrada: Grafo ponderado conexo $G = (V, E)$ **Salida:** Árbol generador mínimo T

```
1:  $T \leftarrow \emptyset$ 
2: Ordenar  $E$  por peso ascendente
3: for cada  $(u, v) \in E$  do
4:   if  $u$  y  $v$  en componentes distintas then
5:      $T \leftarrow T \cup \{(u, v)\}$ 
6:     Unir componentes de  $u$  y  $v$ 
7:   end if
8: end for
9: return  $T$ 
```

8.2. Árboles Generadores Mínimos

8.3. Redes de Transporte

Teorema de Flujo Máximo - Corte Mínimo: el flujo máximo en una red es igual a la capacidad del corte mínimo.

Algoritmo 8 Ford-Fulkerson

Entrada: Red con fuente s , sumidero t , capacidades c

Salida: Flujo máximo

- 1: $\text{flujo}[u][v] \leftarrow 0$ para todo u, v
 - 2: **while** existe camino de aumento P de s a t **do**
 - 3: Encontrar capacidad residual mínima en P
 - 4: Actualizar flujo en P
 - 5: **end while**
 - 6: **return** flujo
-

8.4. Ejercicios

1. Ejecutar Dijkstra en un grafo de ejemplo.
2. Demostrar que Prim encuentra un árbol generador mínimo.
3. Implementar Kruskal en Haskell.

Parte III

Criptografía

Capítulo 9

Aritmética Modular

9.1. Introducción

$a \equiv b \pmod{n}$ si n divide a $a - b$.

Propiedades: reflexiva, simétrica, transitiva.

9.2. Operaciones Modulares

$$a + b \equiv (a \pmod{n}) + (b \pmod{n}) \pmod{n},$$

$$a \cdot b \equiv (a \pmod{n}) \cdot (b \pmod{n}) \pmod{n},$$

$$a^b \pmod{n}.$$

9.3. Algoritmo de Euclides

El algoritmo extendido encuentra x, y tales que $ax + by = \gcd(a, b)$.

9.4. Inversos Modulares

a^{-1} existe si y solo si $\gcd(a, n) = 1$.

Algoritmo 9 Algoritmo de Euclides**Entrada:** $a \geq b > 0$ **Salida:** $\gcd(a, b)$

```

1: while  $b \neq 0$  do
2:    $r \leftarrow a \bmod b$ 
3:    $a \leftarrow b$ 
4:    $b \leftarrow r$ 
5: end while
6: return  $a$ 

```

9.5. Pequeño Teorema de Fermat

Si p es primo y $p \nmid a$:

$$a^{p-1} \equiv 1 \pmod{p}.$$

9.6. Implementación en Haskell

Listing 9.1: Aritmética modular en Haskell

```

1 mcd :: Integral a => a -> a -> a
2 mcd a 0 = abs a
3 mcd a b = mcd b (a `mod` b)
4
5 euclidesExtendido :: Integral a => a -> a -> (a, a, a)
6 euclidesExtendido a 0 = (a, 1, 0)
7 euclidesExtendido a b =
8     let (g, x1, y1) = euclidesExtendido b (a `mod` b)
9     in (g, y1, x1 - (a `div` b) * y1)
10
11 inversoModular :: Integral a => a -> a -> Maybe a
12 inversoModular a m =
13     let (g, x, _) = euclidesExtendido a m
14     in if g == 1 then Just (x `mod` m) else Nothing
15
16 potenciaMod :: Integer -> Integer -> Integer -> Integer
17 potenciaMod base 0 _ = 1
18 potenciaMod base exp mod

```

```
19 | even exp = let p = potenciaMod base (exp `div` 2)
    |         mod
20 |         in (p*p) `mod` mod
21 | otherwise = (base * potenciaMod base (exp - 1) mod)
    |         `mod` mod
```

9.7. Ejercicios

1. Calcular el inverso modular de 3 módulo 11.
2. Demostrar el Pequeño Teorema de Fermat.
3. Implementar en Haskell una función que verifique si un número es primo usando el test de Fermat.

Capítulo 10

Criptografía Clásica

10.1. Introducción

- **Criptografía simétrica:** misma clave para cifrar y descifrar.
- **Criptografía asimétrica:** claves diferentes.

10.2. Cifrado César

Desplaza cada letra un número fijo de posiciones. Con desplazamiento 3:

HOLA $\rightarrow$ KROD.

Solo tiene 25 desplazamientos: es vulnerable a fuerza bruta.

10.3. Cifrado de Sustitución

Reemplaza cada letra por otra según una tabla. El **análisis de frecuencia** permite romperlo.

10.4. Cifrado de Vigenère

Usa una palabra clave para desplazar cada letra de forma diferente. Con clave CLAVE:

HOLA $\rightarrow$ JPRD.

10.5. Implementación en Haskell

Listing 10.1: Criptografía clásica en Haskell

```
1 import Data.Char
2
3 cesar :: Int -> String -> String
4 cesar n = map (cifrar n)
5     where
6         cifrar n c
7             | isAlpha c = chr (ord 'A' + ((ord (toUpper c) -
8                 ord 'A' + n) `mod` 26))
9             | otherwise = c
10
11 cesarFuerzaBruta :: String -> [String]
12 cesarFuerzaBruta texto = [cesar n texto | n <- [0..25]]
13
14 vigenere :: String -> String -> String
15 vigenere clave texto =
16     zipWith cifrar (map toUpper texto) (cycle (map
17         toUpper clave))
18     where
19         cifrar c k = chr (ord 'A' + ((ord c - ord 'A' + ord k
20             - ord 'A') `mod` 26))
```

10.6. Ejercicios

1. Cifrar y descifrar con César.
2. Romper un cifrado César por fuerza bruta.
3. Implementar Vigenère en Haskell.

Capítulo 11

Criptografía RSA

11.1. Introducción

RSA fue desarrollado por Rivest, Shamir y Adleman en 1977. Utiliza dos claves: pública (e, n) y privada (d, n) .

11.2. Generación de Claves

1. Elegir primos grandes p, q .
2. Calcular $n = p \cdot q$.
3. Calcular $\varphi(n) = (p - 1)(q - 1)$.
4. Elegir e con $1 < e < \varphi(n)$ y $\gcd(e, \varphi(n)) = 1$.
5. Calcular $d = e^{-1} \text{ mód } \varphi(n)$.

11.3. Cifrado y Descifrado

Cifrado: $C = M^e \text{ mód } n$. Descifrado: $M = C^d \text{ mód } n$.
Corrección: $M^{ed} \equiv M \text{ (mód } n)$ (teorema de Euler).

11.4. Seguridad

La seguridad de RSA se basa en la dificultad de factorizar $n = p \cdot q$.

11.5. Implementación en Haskell

Listing 11.1: RSA en Haskell

```

1  euclidesExt :: Integer -> Integer -> (Integer, Integer,
    Integer)
2  euclidesExt a 0 = (a, 1, 0)
3  euclidesExt a b =
4      let (g, x1, y1) = euclidesExt b (a `mod` b)
5      in (g, y1, x1 - (a `div` b) * y1)
6
7  inversoMod :: Integer -> Integer -> Maybe Integer
8  inversoMod a m =
9      let (g, x, _) = euclidesExt a m
10     in if g == 1 then Just (x `mod` m) else Nothing
11
12  potenciaMod :: Integer -> Integer -> Integer -> Integer
13  potenciaMod base 0 _ = 1
14  potenciaMod base exp mod
15      | even exp = let p = potenciaMod base (exp `div` 2)
16                  in (p*p) `mod` mod
17      | otherwise = (base * potenciaMod base (exp - 1) mod)
18                  `mod` mod
19
20  generarRSA :: Integer -> Integer -> Integer -> ((Integer,
    Integer),(Integer,Integer))
21  generarRSA p q e =
22      let n = p * q
23          phi = (p-1)*(q-1)
24          d = case inversoMod e phi of
25              Just d' -> d'
26              Nothing -> error "e no invertible"
27      in ((e,n),(d,n))
28
29  cifrarRSA :: Integer -> (Integer,Integer) -> Integer
30  cifrarRSA m (e,n) = potenciaMod m e n
31
32  descifrarRSA :: Integer -> (Integer,Integer) -> Integer
33  descifrarRSA c (d,n) = potenciaMod c d n

```

11.6. Ejercicios

1. Generar un par de claves RSA con primos pequeños.
2. Cifrar y descifrar un mensaje.
3. Implementar una función que convierta texto a números para RSA.

Parte IV

Programación Funcional con Haskell

Capítulo 12

Introducción a Haskell

12.1. ¿Qué es Haskell?

Haskell es un lenguaje funcional puro, de propósito general. Se utiliza en verificación de hardware, telecomunicaciones, comercio electrónico, etc.

12.1.1. Características principales

- **Transparencia referencial:** el resultado depende solo de los argumentos.
- **Funciones como ciudadanos de primera clase:** se pasan, devuelven y almacenan.
- **Evaluación perezosa:** se evalúa solo cuando se necesita.
- **Tipos fuerte y estático:** verificación en compilación.
- **Inferencia de tipos.**
- **Polimorfismo paramétrico y sobrecarga.**

Característica	Int	Integer
Tamaño	Fijo	Arbitrario
Rango	Limitado	Ilimitado
Rendimiento	Rápido	Lento
Riesgo	Desbordamiento	Ninguno

Cuadro 12.1: Comparación entre Int e Integer

12.2. Tipos Básicos

12.2.1. Enteros

12.2.2. Reales

Float (7 dígitos), Double (15-16 dígitos).

12.2.3. Bool, Char, String

Bool: True, False. Char: 'a'. String es [Char].

12.3. Definición de funciones

```

1 areaRectangulo :: Int -> Int -> Int
2 areaRectangulo b h = b * h
3
4 maximo :: Int -> Int -> Int
5 maximo x y | x <= y = y
6             | otherwise = x
7
8 factorial :: Integer -> Integer
9 factorial 0 = 1
10 factorial n = n * factorial (n - 1)

```

12.4. Listas

```

1 [1,2,3] :: [Int]

```

```
2 [1..10] -- [1,2,3,4,5,6,7,8,9,10]
3 [1,3..10] -- [1,3,5,7,9]
4
5 dobles :: [Int] -> [Int]
6 dobles xs = [2*x | x <- xs]
7
8 map :: (a -> b) -> [a] -> [b]
9 map (+1) [1,2,3] -- [2,3,4]
```

12.5. Funciones de Orden Superior

```
1 dosVeces :: (a -> a) -> a -> a
2 dosVeces f x = f (f x)
3
4 (.) :: (b -> c) -> (a -> b) -> (a -> c)
5 (g . f) x = g (f x)
```

12.6. Tuplas

```
1 (1, "hola") :: (Int, String)
2 fst (1, "hola") -- 1
3 snd (1, "hola") -- "hola"
```

12.7. Ejercicios

(Ver el texto completo.)

Capítulo 13

Haskell para Combinatoria

13.1. Generación de Permutaciones

Listing 13.1: Permutaciones en Haskell

```
1 permutaciones :: [a] -> [[a]]
2 permutaciones [] = [[]]
3 permutaciones xs = [ y:ys | y <- xs, ys <- permutaciones
4   (eliminar y xs) ]
5   where
6     eliminar _ [] = []
7     eliminar y (x:xs') | y == x = xs'
8                           | otherwise = x : eliminar y xs'
9
10 combinaciones :: Int -> [a] -> [[a]]
11 combinaciones 0 _ = [[]]
12 combinaciones _ [] = []
13 combinaciones k (x:xs) = map (x:) (combinaciones (k-1) xs)
14                          ++ combinaciones k xs
15
16 subconjuntos :: [a] -> [[a]]
17 subconjuntos [] = [[]]
18 subconjuntos (x:xs) = subconjuntos xs ++ map (x:) (
19   subconjuntos xs)
```

13.2. Números Especiales

```
1 catalan :: Integer -> Integer
2 catalan n = product [n+2..2*n] 'div' product [1..n]
3
4 stirling2 :: Integer -> Integer -> Integer
5 stirling2 0 0 = 1
6 stirling2 _ 0 = 0
7 stirling2 0 _ = 0
8 stirling2 n k = k * stirling2 (n-1) k + stirling2 (n-1) (
    k-1)
```

Capítulo 14

Haskell para Grafos

14.1. Representación de Grafos

Listing 14.1: Representación de grafos en Haskell

```
1 type Vertice = Int
2 type Grafo = [(Vertice, [Vertice])]
3 type GrafoPonderado = [(Vertice, [(Vertice, Int)])]
4
5 grafoDesdeAristas :: [Vertice] -> [(Vertice,Vertice)] ->
   Grafo
6 grafoDesdeAristas vs aristas =
7     [(v, [w | (a,w) <- aristas, a == v] ++ [w | (w,a) <-
8         aristas, a == v]) | v <- vs]
9
10 grafoDirigido :: [Vertice] -> [(Vertice,Vertice)] ->
   Grafo
11 grafoDirigido vs aristas =
    [(v, [w | (a,w) <- aristas, a == v]) | v <- vs]
```

14.2. Algoritmos de Recorrido

Listing 14.2: DFS y BFS en Haskell

```
1 dfs :: Vertice -> Grafo -> [Vertice]
2 dfs inicio grafo = dfsAux inicio []
```


Capítulo 15

Haskell para Criptografía

15.1. Aritmética Modular

Listing 15.1: Aritmética modular en Haskell

```
1 euclidesExt :: Integer -> Integer -> (Integer, Integer,
   Integer)
2 euclidesExt a 0 = (a, 1, 0)
3 euclidesExt a b =
4     let (g, x1, y1) = euclidesExt b (a `mod` b)
5     in (g, y1, x1 - (a `div` b) * y1)
6
7 inversoMod :: Integer -> Integer -> Maybe Integer
8 inversoMod a m =
9     let (g, x, _) = euclidesExt a m
10    in if g == 1 then Just (x `mod` m) else Nothing
11
12 potenciaMod :: Integer -> Integer -> Integer -> Integer
13 potenciaMod base 0 _ = 1
14 potenciaMod base exp mod
15     | even exp = let p = potenciaMod base (exp `div` 2)
16                 in (p*p) `mod` mod
17     | otherwise = (base * potenciaMod base (exp - 1) mod)
   `mod` mod
```

15.2. RSA Completo

Listing 15.2: RSA en Haskell

```
1  generarRSA :: Integer -> Integer -> Integer -> ((Integer ,
   Integer), (Integer, Integer))
2  generarRSA p q e =
3      let n = p*q
4          phi = (p-1)*(q-1)
5          d = case inversoMod e phi of
6              Just d' -> d'
7              Nothing -> error "no invertible"
8      in ((e,n), (d,n))
9
10 textoANumero :: String -> Integer
11 textoANumero = foldl (\acc c -> acc * 256 + fromIntegral
   (ord c)) 0
12
13 numeroATexto :: Integer -> String
14 numeroATexto 0 = ""
15 numeroATexto n = let (q,r) = n `divMod` 256
16                  in numeroATexto q ++ [chr (fromIntegral
   r)]
```

Parte V

Guías de Práctica

Capítulo 16

Guía de Práctica 1: Técnicas Avanzadas de Conteo

16.1. Objetivos

- Aplicar los principios básicos de combinatoria.
- Establecer y resolver relaciones de recurrencia.
- Utilizar funciones generatrices.
- Analizar la complejidad de algoritmos.

16.2. Principios Básicos de Combinatoria

Ejercicio 16.1. ¿Cuántos caracteres Braille distintos puede haber?

Ejercicio 16.2. ¿Cuántas placas de automóvil con 3 letras seguidas de 2 dígitos?

Ejercicio 16.3. Manos de póquer de 5 cartas: cuatro de un tipo, escalera de color, $2+2+1$.

Ejercicio 16.4. Moneda lanzada 10 veces: ¿cuántos resultados con 3 caras?

Ejercicio 16.5. Cadenas de 8 bits con a lo más 3 ceros.

Ejercicio 16.6. Comité de 4 republicanos, 3 demócratas, 2 independientes.

Ejercicio 16.7. Palabras clave de 4 caracteres.

Ejercicio 16.8. Ordenar 4 libros de matemáticas, 6 de física y 2 de química.

Ejercicio 16.9. Cadenas con letras ABCDEF bajo condiciones.

Ejercicio 16.10. Repartir 25 rosquillas entre 4 personas con restricciones.

16.3. Relaciones de Recurrencia

Ejercicio 16.11. R_n = regiones por n líneas. Calcular R_1, R_2, R_3 y relación de recurrencia.

Ejercicio 16.12. S_n = cadenas de n bits con $C(\alpha) \leq 2$.

Ejercicio 16.13. Resolver: $2a_n = 7a_{n-1} - 3a_{n-2} + 2^n$, etc.

16.4. Funciones Generatrices

Ejercicio 16.14. Asignar dos docenas de robots a 4 líneas de ensamble.

Ejercicio 16.15. 300 sobres en paquetes de 25 entre 4 grupos.

Ejercicio 16.16. Distribuir n objetos en k cajas con al menos 1.

Ejercicio 16.17. Elegir 5 frutas entre manzanas, naranjas y peras.

16.5. Análisis de Algoritmos

Ejercicio 16.18. Notación Θ para $(n+1)(n+3)$ y $2 \log n + 4n + 5n \log n$.

Ejercicio 16.19. Orden de los bucles anidados.

Ejercicio 16.20. Algoritmo recursivo máximo/mínimo: relación de recurrencia.

Ejercicio 16.21. Ordenamiento por inserción: peor caso.

Ejercicio 16.22. Evaluación de polinomios.

Ejercicio 16.23. Exponenciación rápida: complejidad $\Theta(\log n)$.

16.6. Problemas Integradores

Ejercicio 16.24. Demuestre que el número de formas de distribuir n objetos idénticos en k cajas distintas es $\binom{n+k-1}{k-1}$.

Ejercicio 16.25. Analice la complejidad del ordenamiento por inserción ($O(n^2)$).

16.7. Referencias

- Richard Johnsonbaugh, *Matemáticas Discretas*, 6^a ed.
- Kenneth H. Rosen, *Discrete Mathematics and Its Applications*, 7^a ed.

Capítulo 17

Práctica: Grafos en Haskell

17.1. Objetivos

Al finalizar esta práctica, el estudiante será capaz de:

- Implementar representaciones de grafos en Haskell.
- Implementar algoritmos de recorrido y búsqueda.
- Implementar algoritmos de caminos mínimos.

17.2. Ejercicios de Implementación

17.2.1. Representación y Recorridos

1. Implementar una función que convierta una matriz de adyacencia a una lista de adyacencia.
2. Implementar DFS y BFS usando colas y pilas explícitas.
3. Implementar una función que verifique si un grafo es conexo.
4. Implementar una función que encuentre todas las componentes conexas.

17.2.2. Algoritmos Avanzados

1. Implementar el algoritmo de Dijkstra usando una cola de prioridad.

2. Implementar el algoritmo de Prim.
3. Implementar el algoritmo de Kruskal usando Union-Find.
4. Implementar una función que encuentre un ciclo euleriano.

17.3. Proyecto

Desarrollar una librería en Haskell para teoría de grafos con las siguientes funcionalidades:

- Representación de grafos (no dirigidos, dirigidos, ponderados).
- Algoritmos de recorrido (DFS, BFS).
- Algoritmos de caminos mínimos (Dijkstra, Floyd-Warshall).
- Algoritmos de árboles generadores (Prim, Kruskal).
- Verificación de propiedades (conexidad, bipartismo, planaridad).

Capítulo 18

Práctica: Criptografía en Haskell

18.1. Objetivos

Al finalizar esta práctica, el estudiante será capaz de:

- Implementar cifrados clásicos en Haskell.
- Implementar aritmética modular y criptografía RSA.
- Analizar la seguridad de diferentes sistemas criptográficos.

18.2. Ejercicios de Implementación

18.2.1. Cifrados Clásicos

1. Implementar el cifrado César con diferentes alfabetos.
2. Implementar el cifrado de Vigenère.
3. Implementar un ataque de análisis de frecuencia.
4. Implementar el cifrado de transposición de columnas.

18.2.2. RSA

1. Implementar la generación de claves RSA.
2. Implementar cifrado y descifrado RSA.

3. Implementar conversión texto-número para RSA.
4. Implementar un ataque de factorización para claves pequeñas.

18.3. Proyecto

Desarrollar un sistema de criptografía completo en Haskell que incluya:

- Módulo de aritmética modular.
- Cifrados clásicos (César, Vigenère).
- Cifrado RSA con generación de claves.
- Interfaz de línea de comandos para uso práctico.